Лабораторна робота №4

З дисципліни “Системи програмування”

На тему: “Асинхронна модель програмування (TPL). Кінцеві автомати (async/await), звітування про прогрес та кооперативне скасування“

Студента 4 курсу: Групи
МІТ-41 Демиденко Андрій

Київ - 2026р.

1. МЕТА ТА ЗАГАЛЬНА АРХІТЕКТУРА ПРОЕКТУ

Мета: Розробити програмний комплекс для дослідження асинхронної моделі програмування на базі Task Parallel Library (TPL). Довести ефективність використання кінцевих автоматів (`async/await`), механізмів делегування фонових задач (`Task.Run`) та кооперативного скасування потоків для збереження чуйності графічного інтерфейсу користувача (UI) під час виконання обчислювально містких (CPU-bound) операцій.

Загальна архітектура: Проект реалізовано як десктопний графічний додаток. Архітектура базується на жорсткому розділенні потоків:

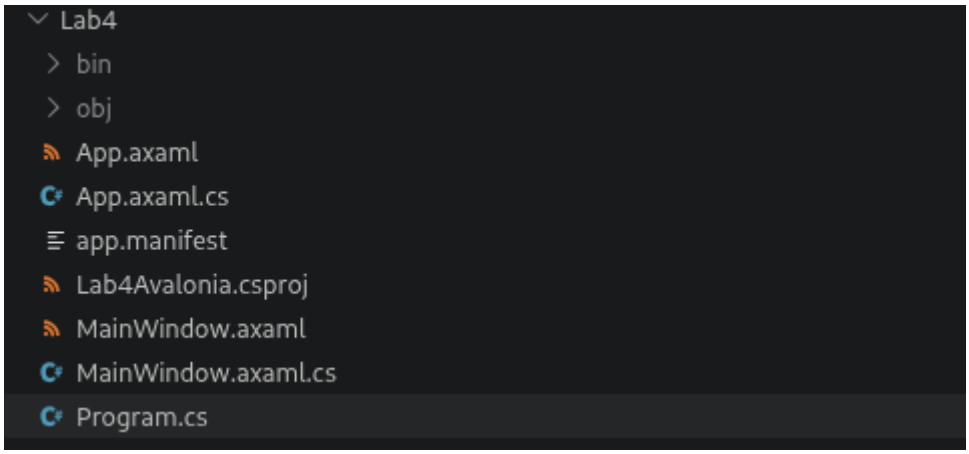
1. **UI Thread (Головний потік):** Відповідає виключно за рендеринг інтерфейсу та обробку подій вводу.
2. **ThreadPool (Пул робочих потоків):** Використовується для виконання тривалих математичних обчислень у фоновому режимі. Синхронізація між потоками (передача прогресу та сигналів скасування) здійснюється через потокобезпечні абстракції `IProgress<T>` та `CancellationToken`.

2. АРХІТЕКТУРНЕ ОБҐРУНТУВАННЯ: ВИБІР ФРЕЙМВОРКУ AVALONIA UI

Для повноцінної демонстрації теми "Асинхронне програмування" та проблеми "заморожування" інтерфейсу (UI Freezing) консольного середовища недостатньо. Було прийнято рішення про розробку графічного інтерфейсу. Вибір фреймворку **Avalonia UI** як основи проекту зумовлений наступними інженерними факторами:

1. **Кросплатформена сумісність (Debian Linux):** Класичні технології Microsoft для побудови UI (Windows Forms, WPF, UWP) мають жорстку залежність від графічних підсистем Windows (DirectX, User32). Оскільки розробка та тестування проводяться в середовищі Debian Linux, їх використання неможливе. Avalonia UI використовує графічний рушій Skia, що дозволяє нативно рендерити інтерфейс через X11 або Wayland без використання емуляторів чи проміжних шарів (Wine/Mono).
2. **Ізоляція контексту синхронізації (Dispatcher):** Avalonia має строгу однопотокову модель UI (Dispatcher.UIThread). Це створює ідеальне середовище для тестування механізмів TPL: будь-яка спроба синхронно виконати важку задачу на кнопці миттєво блокує рендеринг, а спроба оновити `ProgressBar` з фонового потоку викликає виняток безпеки (`InvalidOperationException`). Це дозволяє на практиці застосувати та обґрунтувати необхідність використання `IProgress<T>`.

3. **Сучасний технологічний стек:** На відміну від застарілих рішень, Avalonia повністю підтримує сучасний C# (версії 10-12), сувору перевірку Nullable-типів та механізм Compiled Bindings (x:DataType). Це мінімізує помилки етапу виконання (Runtime Errors) та забезпечує високу продуктивність застосунку.



3. РЕАЛІЗАЦІЯ БАЗОВИХ МЕХАНІЗМІВ АСИНХРОННОСТІ (ЗАВДАННЯ 1-2)

Проблема: Виконання циклу з мільйонами ітерацій в обробнику події `OnClick` призводить до блокування циклу повідомлень віконного менеджера. Додаток перестає реагувати на дії користувача, а ОС Linux може запропонувати "Примусово закрити" програму.

Рішення (async/await + Task.Run): Обчислювальну логіку винесено в окремий метод, який викликається через `Task.Run()`. Цей метод інструктує планувальник .NET виділити вільний потік з `ThreadPool` для виконання коду. Обробник події `OnClick` позначено модифікатором `async`. Виклик фонової задачі супроводжується оператором `await`. Це перетворює метод на кінцевий автомат: головний потік UI не чекає завершення обчислень, а негайно повертається до відмальовування вікна. Після завершення роботи фонового потоку, виконання обробника автоматично відновлюється з наступного рядка коду.

```
private Task<int> ProcessLongTaskAsync(int totalSteps, IProgress<int> progress, CancellationToken token)
{
    return Task.Run(() =>
    {
        int sum = 0;

        for (int i = 1; i <= totalSteps; i++)
        {
            // Перевірка стану маркера. Якщо викликано Cancel(), генерується виняток.
            token.ThrowIfCancellationRequested();

            Thread.Sleep(100);
            sum += i;

            // Відправка проміжного результату до головного потоку
            progress?.Report(i);
        }

        return sum;
    }, token);
}
```

4. МАРШРУТИЗАЦІЯ ПРОГРЕСУ ТА СКАСУВАННЯ (ЗАВДАННЯ 3-4)

Звітування про прогрес (IProgress): Для оновлення індикатора завантаження (`ProgressBar`) в Avalonia використано клас `Progress<int>`. Він ініціалізується в головному потоці (захоплюючи його `SynchronizationContext`) і передається як інтерфейс у фонову задачу. Коли робочий потік викликає `progress.Report(value)`, TPL автоматично маршалізує цей виклик у чергу `Dispatcher` головного потоку, що гарантує потокобезпечне оновлення UI без виникнення помилок міжпотокового доступу.

Кооперативне скасування (CancellationToken): Апаратне знищення потоків (`Thread.Abort()`) порушує цілісність пам'яті програми. У проекті реалізовано патерн кооперативного скасування. Генерується екземпляр `CancellationTokenSource`, з якого вилучається `Token` і передається у фоновий цикл. При натисканні кнопки "Скасувати" викликається метод `Cancel()`. Фоновий потік на кожній ітерації перевіряє стан токена (`ThrowIfCancellationRequested()`). Виявивши сигнал скасування, потік коректно зупиняє розрахунки і генерує виняток `OperationCanceledException`, який перехоплюється блоком `try/catch` у головному потоці.

```
_cancellationTokenSource = new CancellationTokenSource();

// Ініціалізація диспетчера прогресу у головному потоці (UI Thread)
var progressHandler = new Progress<int>(percent =>
{
    PbTaskProgress.Value = percent;
    TxtProgressPercent.Text = $"{percent}%";
});
```

ВИСНОВОК

У рамках лабораторної роботи розроблено повноцінний програмний продукт, що об'єднує механізми апаратно-незалежного графічного рендерингу (Avalonia UI) та високорівневі абстракції багатопотоковості (Task Parallel Library).

Доведено, що використання класичних платформно-залежних UI-фреймворків є недоцільним при розробці системного та прикладного ПЗ для Linux (Debian). Вибір Avalonia дозволив імплементувати строгу модель ізоляції потоків (Dispatcher), на якій успішно протестовано концепцію неблокуючого вводу-виводу (Non-blocking I/O) та фонових CPU-bound обчислень за допомогою операторів `async/await`.

Впровадження об'єктів `IProgress<T>` та `CancellationToken` продемонструвало інженерно правильний підхід до міжпотокової взаємодії, гарантуючи чуйність (Responsiveness) графічного інтерфейсу та безпечне вивільнення ресурсів при достроковому перериванні операцій користувачем. Отримана архітектура є масштабованою і відповідає сучасним стандартам розробки .NET-додатків індустріального рівня.